

Smart Disaster Response MIS

Transaction Handling Demonstration

DELIVERABLE — ACID TRANSACTIONS

Course: Database Systems

Campus: FAST-NUCES, Islamabad

Semester: Spring 2026

Document: Transaction Handling

Table of Contents

1. Overview & ACID Compliance
2. Standard Transaction Pattern
3. Transaction Case Studies
 - 3.1 Resource Dispatch
 - 3.2 Team Assignment
 - 3.3 Donation Recording
 - 3.4 Patient Admission
4. Concurrency & Isolation
5. Summary

1. Overview & ACID Compliance

Property	Implementation
Atomicity	BEGIN TRANSACTION / COMMIT / ROLLBACK guarantees all-or-nothing semantics for every multi-table operation.
Consistency	CHECK constraints, trigger invariants, and referential integrity prevent the database entering an invalid state.
Isolation	SQL Server's default READ COMMITTED isolation prevents dirty reads across concurrent transactions.
Durability	COMMIT writes are persisted to the transaction log before acknowledgement, surviving system failures.

Table 1 — ACID property implementation

2. Standard Transaction Pattern

All transactions follow the SQL Server structured error-handling pattern below. Any error — constraint violation, trigger rollback, or runtime error — produces a clean rollback and a meaningful error message.

```
BEGIN TRY
BEGIN TRANSACTION;

-- Step 1: First table modification
UPDATE TableA
SET column1 = @value1
WHERE id = @id;

-- Step 2: Dependent table modification
INSERT INTO TableB (col_a, col_b)
VALUES (@val_a, @val_b);

COMMIT TRANSACTION;
SELECT 'SUCCESS' AS result;

END TRY
BEGIN CATCH
IF @@TRANCOUNT > 0
ROLLBACK TRANSACTION;

SELECT
ERROR_NUMBER() AS error_code,
ERROR_MESSAGE() AS error_message,
ERROR_SEVERITY() AS severity;
END CATCH;
```

Listing 1 — Standard ACID transaction pattern

3. Transaction Case Studies

3.1 Resource Dispatch

Dispatching a resource atomically updates the allocation status to '**dispatched**' and deducts stock from **WarehouseInventory**. If stock is insufficient, the trigger raises an error and the entire transaction rolls back, preventing ghost dispatches.

Tables & Conditions Involved

- ResourceAllocations — status changed to 'dispatched'
- WarehouseInventory — quantity_available decremented
- Rollback condition: quantity_available would become negative

Implementation

```
BEGIN TRY
BEGIN TRANSACTION;

UPDATE ResourceAllocations
SET status = 'dispatched',
    dispatched_at = GETDATE()
WHERE allocation_id = @allocation_id;

-- trg_ResourceAllocation_Dispatch fires here;
-- rolls back if stock would go negative.

COMMIT TRANSACTION;
SELECT 'Resource dispatched successfully.' AS result;

END TRY
BEGIN CATCH
IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
SELECT ERROR_MESSAGE() AS error_message;
END CATCH;
```

Listing — 3.1 Resource Dispatch transaction

3.2 Team Assignment

Assigning a team requires inserting the **TeamAssignment** record and updating the report status to '**in_progress**'. An assignment without a status update would leave the report appearing idle.

Tables & Conditions Involved

- TeamAssignments — new assignment record inserted
- EmergencyReports — status updated to 'in_progress'
- Rollback condition: either operation fails

Implementation

```
BEGIN TRY
BEGIN TRANSACTION;

INSERT INTO TeamAssignments
(report_id, team_id, assigned_at, assigned_by)
VALUES (@report_id, @team_id, GETDATE(), @operator_id);

UPDATE EmergencyReports
SET status = 'in_progress',
operator_id = @operator_id
WHERE report_id = @report_id;

COMMIT TRANSACTION;
SELECT 'Team assigned successfully.' AS result;

END TRY
BEGIN CATCH
IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
SELECT ERROR_MESSAGE() AS error_message;
END CATCH;
```

Listing — 3.2 Team Assignment transaction

3.3 Donation Recording

Every donation simultaneously creates a **Donations** record and a **FinanceTransactions** ledger entry. A donation without a transaction record produces an inconsistent financial ledger.

Tables & Conditions Involved

- Donations — donor record inserted
- FinanceTransactions — corresponding ledger entry inserted
- Rollback condition: either insert fails

Implementation

```
BEGIN TRY
BEGIN TRANSACTION;

INSERT INTO Donations
(donor_name, amount, donated_at, campaign_id)
VALUES (@donor_name, @amount, GETDATE(), @campaign_id);

INSERT INTO FinanceTransactions
(transaction_type, amount, transaction_timestamp,
reference_id, recorded_by)
VALUES ('donation', @amount, GETDATE(),
SCOPE_IDENTITY(), @user_id);

COMMIT TRANSACTION;
```

```
SELECT 'Donation recorded successfully.' AS result;

END TRY
BEGIN CATCH
IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
SELECT ERROR_MESSAGE() AS error_message;
END CATCH;
```

Listing — 3.3 Donation Recording transaction

3.4 Patient Admission

Admitting a patient inserts the patient record and decrements **Hospitals.available_beds**. The **trg_Patient_Admission** trigger rolls back the transaction if no beds are available.

Tables & Conditions Involved

- Patients — admission record inserted
- Hospitals — available_beds decremented by 1
- Rollback condition: available_beds is already zero

Implementation

```
BEGIN TRY
BEGIN TRANSACTION;

INSERT INTO Patients
(report_id, hospital_id, full_name,
condition, admitted_at)
VALUES (@report_id, @hospital_id, @full_name,
@condition, GETDATE());

-- trg_Patient_Admission fires:
-- decrements beds; ROLLBACK if beds = 0.

COMMIT TRANSACTION;
SELECT 'Patient admitted successfully.' AS result;

END TRY
BEGIN CATCH
IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
SELECT ERROR_MESSAGE() AS error_message;
END CATCH;
```

Listing — 3.4 Patient Admission transaction

4. Concurrency & Isolation

Under **READ COMMITTED** isolation, in high-contention scenarios — e.g., two operators simultaneously dispatching the last unit of a critical resource — **trg_ResourceAllocation_Dispatch** acts as the final gatekeeper. Only one dispatch succeeds within its transaction; the other is rolled back with a meaningful error, even if the application-layer stock check passed for both.

5. Summary

Transaction	Atomicity Guarantee	Trigger Involved
Resource Dispatch	Allocation status + stock deduction	trg_ResourceAllocation_Dispatch
Team Assignment	Assignment record + report status	trg_TeamAssignment_Insert
Donation Recording	Donor record + financial ledger	trg_FinanceTransaction_AuditLog
Patient Admission	Patient record + bed decrement	trg_Patient_Admission

Table 2 — Transaction summary